

[T3] Datapath e execução¹

Caroline Marques Lau¹, Victória Stephanie Mottin²

Universidade Tecnológica Federal do Paraná – UTFPR

COCIC – Coordenação do Curso de Bacharelado em Ciência da Computação

Campo Mourão, Paraná, Brasil

¹carolinelau@alunos.utfpr.edu.br

²vmottin@alunos.utfpr.edu.br

¹ Trabalho desenvolvido para a disciplina de BCC2003 – Arquitetura e Organização de Computadores

Resumo

Um datapath dentro de um processador MIPS representa o caminho que cada instrução realiza para ser decodificada e executada. Esse caminho de dados conta com os seguintes componentes: um contador de programa, uma memória de instruções e outra de dados, um decodificador, um banco de registradores, um extensor de sinal, uma unidade lógica e aritmética, uma unidade de controle e uma sequência de multiplexadores que auxiliam no controle e execução dos diferentes tipos de instruções. Com a finalidade de detalhar e guiar o funcionamento de um datapath de um processador MIPS, esse trabalho representa a parte final da construção desse circuito. Através da plataforma do Logisim, foram simulados seis componentes: um banco com 32 registradores, uma unidade lógica e aritmética (ULA), um decodificador, uma unidade de controle principal (UC), uma unidade de controle da ULA e uma unidade de extensão de sinal.

1. Introdução

O caminho de dados de um processador define a busca de instruções, decodificação e leitura do banco de registradores, execução ou cálculo de endereço, acesso à memória de dados e escrita do resultado [1].

Conforme descrito por Patterson e Hennessy (2017), a unidade lógica e aritmética é um dos componentes fundamentais de um *datapath* em um sistema computacional, sendo projetada para garantir precisão em todas as operações necessárias. Paralelamente, o banco de registradores atua como um repositório para dados, reduzindo o tempo utilizado para acessar informações utilizadas pelo processador com maior frequência. Para garantir a coordenação entre seus elementos, o *datapath* integra também um decodificador de instruções, responsável por segmentar a instrução em campos específicos, dividindo os 32 bits da mesma em campos de modo que os sinais gerados direcionem os outros componentes.

A unidade de controle central, conforme ainda destacam os autores, utiliza o Opcode decodificado da instrução para gerar sinais de controle globais, utilizados ao longo de todo o caminho de dados, como RegWrite, ALUSrc e MemtoReg. Ademais, a unidade de controle da

ULA emprega o campo *funct* (em instruções tipo R) para definir a operação exata da unidade lógica e aritmética a ser executada, em conjunto do sinal ALUOp. Já o extensor de sinal atua na expansão de valores imediatos (*imm*) de 16 para 32 bits, preservando o sinal numérico em arquiteturas MIPS, essencial para operações como *beq*, *lw*, *sw* e *addi* [1].

De modo a implementar essas estruturas e analisar seu comportamento, compreendendo conceitos de arquitetura e organização de computadores, o presente artigo está organizado da seguinte forma: a próxima seção descreve a implementação, através do ambiente virtual Logisim, dos componentes mencionados. Dentro desta, estão descritas as operações feitas pela ULA na seção 2.1, bem como o comportamento do banco de registradores na seção 2.2, e suas respectivas entradas, saídas e objetos de controle. Na seção 2.3 encontra-se a descrição do comportamento do decodificador, e da unidade de extensão de sinal na seção 2.4. As seções seguintes expõem as unidades de controle do caminho de dados, sendo a 2.5 a exposição do comportamento da unidade de controle principal (UC) e a 2.6 as especificações da unidade de controle da ULA. Na seção 2.7 estão expostos todos os multiplexadores utilizados ao longo do caminho de dados para direcionamento das instruções, assim como os sinais de controle que operam cada um. Encontra-se na seção 2.8 o circuito do concatenador para definição de endereços em desvios, na 2.9 a Figura com o circuito completo e na 3 um passo a passo do funcionamento do simulador, com exemplos. Finalmente, na seção 4 há o código escolhido pelas autoras para ser executado no caminho de dados, em linguagens C, assembly e de máquina, e seu valor na tabela de pontuações definida pelo professor da disciplina. As considerações finais estão expostas na seção 5, e no final tem-se as referências utilizadas.

2. Implementação

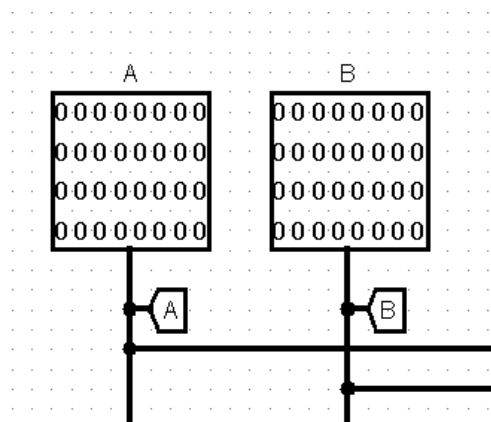
Através do simulador lógico Logisim, foram desenvolvidos pelos autores uma ULA (unidade lógica e aritmética) com 9 operações, um banco de registradores de 32 bits, um decodificador, uma unidade de controle principal (UC), uma unidade de controle da ULA, uma unidade de extensão de sinal e um circuito formado pela união destes elementos, em conjunto com vários multiplexadores para tomada de decisões. Esses módulos foram projetados com o

objetivo de funcionarem em conjunto com outros componentes, concluindo a construção de um caminho de dados monociclo para um subconjunto de instruções da arquitetura MIPS.

2.1. Operações da ULA

A unidade lógica e aritmética é o circuito onde são executadas as operações aritméticas e lógicas dentro de um processador. Ela conta com duas entradas de 32 bits, que são representadas com rótulos A e B (Figura 1), e são utilizadas para a realização de suas operações.

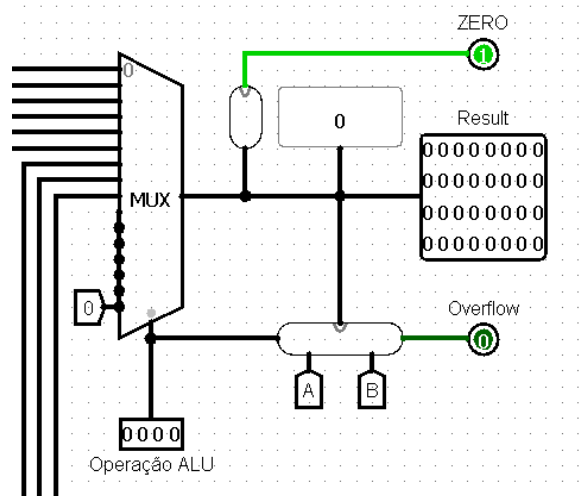
Figura 1 - Entradas A e B



Fonte: Autoria própria, 2024.

A saída da ULA é decidida de acordo com a operação selecionada na entrada de 4 bits Operação ALU, uma vez que ela está conectada a um multiplexador, e é mostrada no pino de saída de 32 bits com rótulo Result (Figura 2).

Figura 2 - Multiplexador e saídas da ULA



Fonte: Autoria própria, 2024.

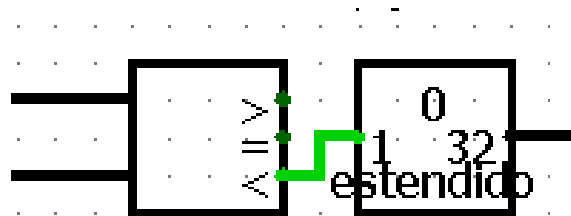
A utilização de 4 bits para seleção de operação através da entrada Operação ALU permite a implementação de até 16 operações diferentes, todavia, para o presente projeto foram escolhidas somente 9. As outras 7 operações faltantes no multiplexador foram, à escolha da equipe, completadas com uma constante 0 (em formato de 32 bits), de modo a retornar este valor ao invés de erro ou indeterminação na saída do multiplexador.

Desta forma, as 9 operações implementadas são:

- **[0000] AND:** Operação lógica bit a bit. O bit em Result será 1 apenas se ambos os bits correspondentes das entradas A e B forem 1.
- **[0001] OR:** Operação lógica bit a bit. O bit em Result será 1 se pelo menos um dos bits correspondentes das entradas A e B for 1.
- **[0010] NOR:** Operação lógica bit a bit. Combina as entradas com a operação OR e inverte o resultado. O bit em Result será 1 apenas se ambos os bits de entrada forem 0.

- **[0011] XOR:** Operação lógica bit a bit. O bit em Result será 1 se os bits correspondentes das entradas A e B forem diferentes (0 e 1, ou 1 e 0).
- **[0100] SLT:** Utilizando o comparador do Logisim, compara os valores das entradas A e B e retorna 1 (com 1 bit) se o primeiro valor for menor que o segundo. Caso contrário, a saída é 0 (com 1 bit).
 - Considerando que todas as entradas do multiplexador precisam ter a mesma quantidade de bits, e a operação SLT é a única das escolhidas que retorna somente 1 bit, foi utilizado um extensor de bits em sua saída antes desta ser ligada a entrada do multiplexador. Assim, na saída do multiplexador, caso essa operação seja escolhida, Result receberá 1 ou 0 em 32 bits (Figura 3).

Figura 3 - Extensor de bits na saída do comparador

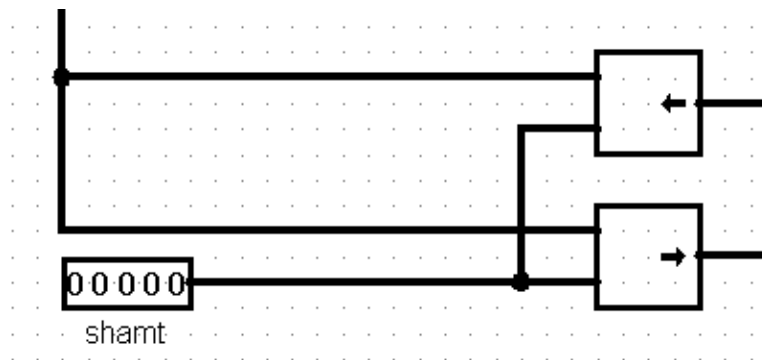


Fonte: Autoria própria, 2024.

- **[0101] ADD:** Através do somador do Logisim, realiza a soma dos valores das entradas A e B, utilizando o conceito de propagação de carry (bit de transporte).
- **[0110] SUB:** Fazendo o uso do subtrator do Logisim, calcula a diferença entre as duas entradas A e B, convertendo a última para o complemento de dois e somando com A.
- **[0111] SLL:** Por meio do deslocador do Logisim, desloca uma quantidade de bits - definida em SHAMT - de A para a direita, preenchendo os bits mais significativos com zeros.

- **[1000] SRL:** Usando o deslocador do Logisim, desloca uma quantidade de bits - definida em *shamt* - de A para a esquerda, preenchendo os bits menos significativos com zeros.
 - O SHAMT (Figura 4), utilizado unicamente nas operações de deslocamento presentes na ULA (SLL e SRL), é uma entrada de 5 bits que define quantos bits de A serão deslocados, seja para esquerda ou direita. Ele está ligado diretamente à entrada Distance dos dois deslocadores.

Figura 4 - Entrada SHAMT



Fonte: Autoria própria, 2024.

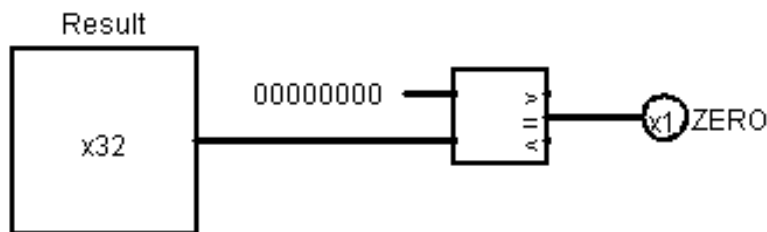
2.1.1. ZERO

O pino ZERO (Figura 5) é uma das três saídas da ULA e está presente para sinalizar quando Result é 0. Dessa forma, caso Result seja 0 (em 32 bits), ZERO possui valor 1 (em 1 bit) e caso Result possua qualquer outro valor, ZERO é 0 (em 1 bit).

Para que funcionasse desta forma, foi criado um circuito separado (Figura 5) que compara uma entrada de 32 bits (Result) com uma constante 0 também em 32 bits utilizando um comparador do Logisim. Conectando um pino à saída Equal do comparador, de igualdade, de acordo com o comportamento do comparador este pino só receberá valor 1 se as duas entradas forem iguais, ou seja, se Result for igual a 0.

Este circuito é conectado à saída Result do circuito da ULA principal, de modo que sua saída seja o pino ZERO.

Figura 5 - Circuito ZERO



Fonte: Autoria própria, 2024.

2.1.2. Overflow

De acordo com Patterson e Hennessy (2017), o *overflow* ocorre quando o resultado de uma operação não pode ser representado com o hardware disponível, nesse caso, uma saída Result de 32 bits. Assim, podem ser armazenados resultados de operações que estão entre -2.147.483.648 e 2.147.483.647 (2^{32} valores), mas quaisquer valores fora desse intervalo causam *overflow*.

A fim de identificar quando o resultado apresentado em Result não está correto, e as operações realizadas na ULA resultaram em *overflow*, foi adicionado um pino de saída de 1 bit nomeado Overflow, que possui valor 1 somente quando essa situação ocorre, e caso contrário, valor 0.

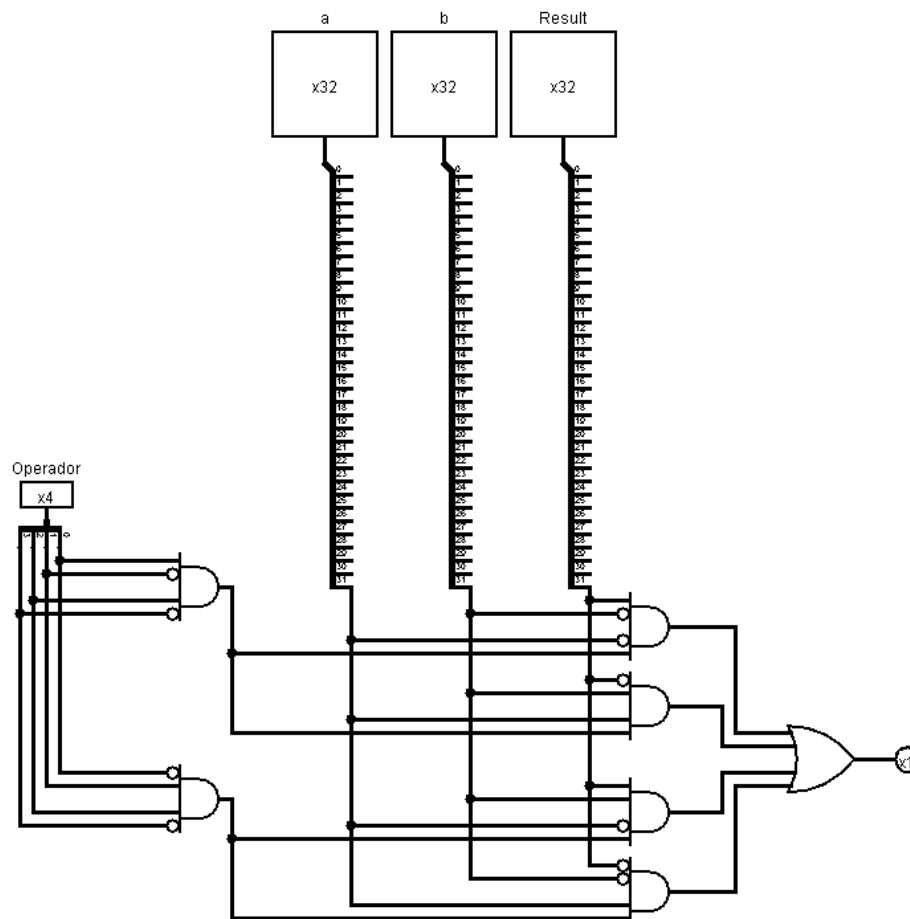
Para que a saída funcionasse corretamente foi identificado que, dentre as operações presentes na ULA, esse “transbordamento” só ocorre nas operações de soma e subtração. Mais especificamente, o *overflow* é identificado quando, em uma soma, as entradas possuem o mesmo sinal e a saída possui sinal diferente de ambas. Já na subtração, o *overflow* ocorre quando as entradas A e B possuem sinais diferentes, e o resultado possui o mesmo sinal que B.

Com base nessas informações foi possível chegar ao circuito overflow (Figura 6), que pega o bit mais significativo de a, b e Result, indicativo de sinal (0 para valores positivos, e 1

para negativos), e realiza as comparações mencionadas através de portas lógicas AND e OR especificamente quando o valor selecionado na entrada Operador é o equivalente às operações de soma ou subtração (operações 0101 e 0110 no multiplexador, respectivamente).

Este circuito é conectado às entradas A, B, Operação ALU no circuito principal da ULA, e à saída Result, de modo que sua saída seja o pino Overflow.

Figura 6 - Circuito overflow

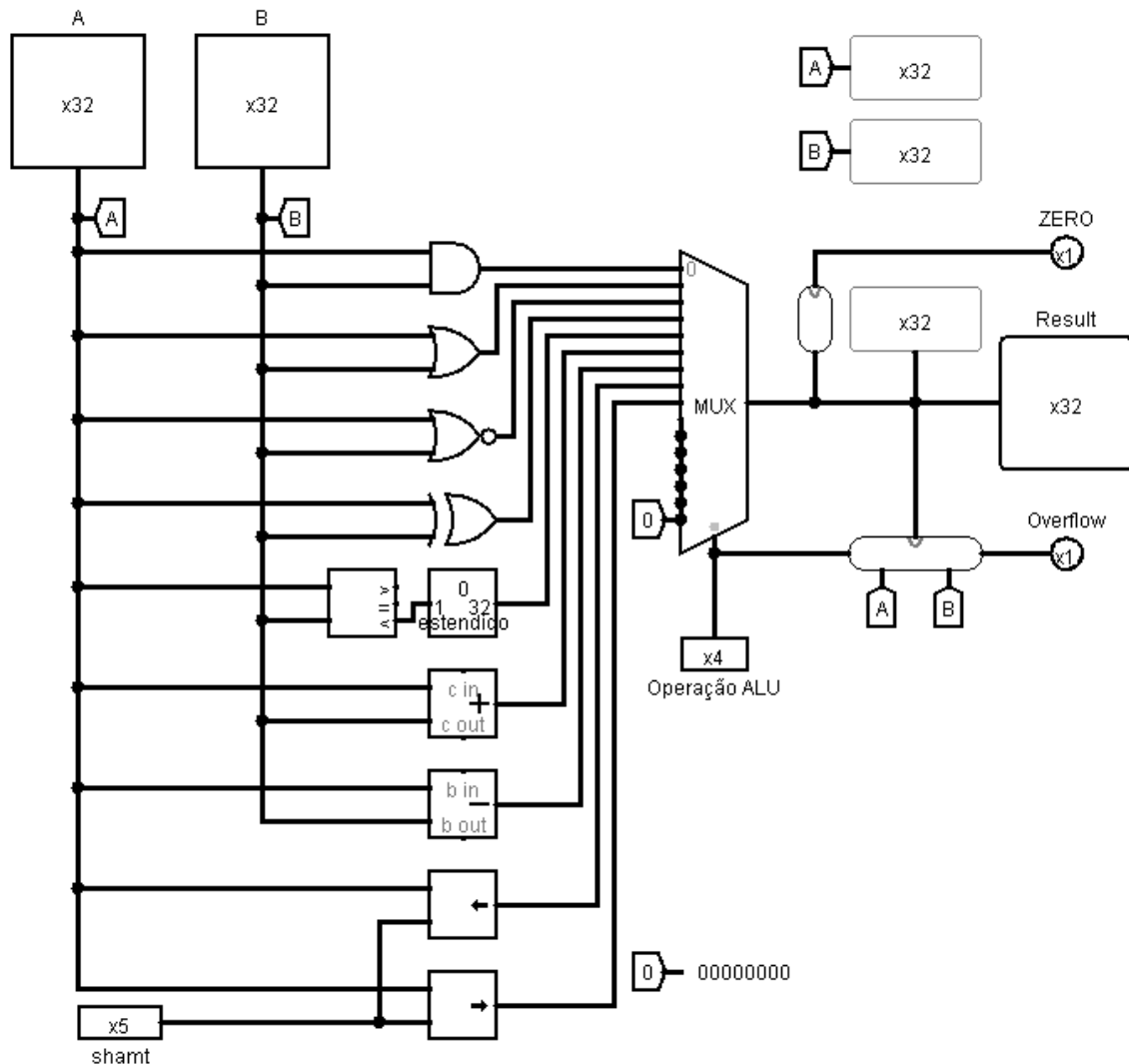


Fonte: Autoria própria, 2024.

Portanto, com base nos componentes listados foi construída a unidade lógica e aritmética no Logisim (Figura 7), de modo que essa realize 9 operações entre os valores das entradas de 32

bits A e B, denote o resultado na saída Result e mostre se houve *overflow* ou se o resultado foi igual a 0, através dos pinos ZERO e Overflow.

Figura 7 - Visão geral da ULA



Fonte: Autoria própria, 2024.

2.2. Comportamento do banco de registradores

Dentro do caminho dos dados da arquitetura de conjunto de instruções MIPS, depois que uma instrução foi decodificada e caso ela pertença aos modos de endereçamento “R” ou “I”, parte

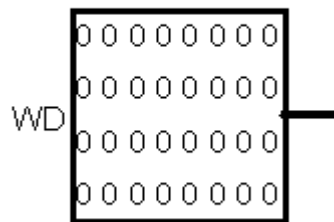
dos bits da instrução são enviados para um componente denominado banco de registradores. Em um processador, esse elemento é composto pelos registradores de uso geral que armazenam dados de forma organizada e permitem leitura ou escrita.

O banco de registradores é um dos componentes de estado do *datapath* de um processador MIPS, pois suas saídas dependem do número dos registradores fornecidos e do conteúdo do estado interno dos registradores que foi escrito anteriormente [1].

Para que fosse possível implementar esse banco no Logisim, definimos alguns pinos:

- **WD (Write Data):** Lugar no qual é inserido um dado de 32 bits a ser escrito no registrador contido em WR (Figura 8).

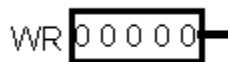
Figura 8 - Pino WD



Fonte: Autoria própria, 2024.

- **WR (Write Register):** Armazena o número do registrador (0-31) no qual um dado será escrito, por isso possui 5 bits (Figura 9).

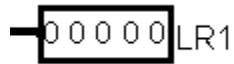
Figura 9 - Pino WR



Fonte: Autoria própria, 2024.

- **LR1 (Load Register 1):** Campo no qual é inserido o número do registrador (0-31) a ser lido na saída LD1, por isso possui 5 bits (Figura 10).

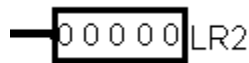
Figura 10 - Pino LR1



Fonte: Autoria própria, 2024.

- **LR2 (Load Register 2):** Campo no qual é inserido o número do registrador (0-31) a ser lido na saída LD2, por isso possui 5 bits (Figura 11).

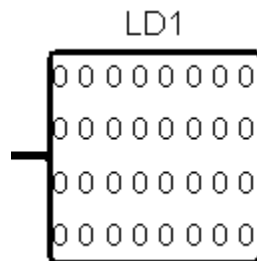
Figura 11 - Pino LR2



Fonte: Autoria própria, 2024.

- **LD1 (Load Data 1):** Saída que retorna o valor (32 bits) armazenado no registrador indicado em LR1 (Figura 12).

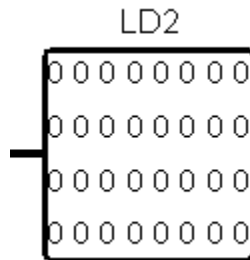
Figura 12 - Pino LD1



Fonte: Autoria própria, 2024.

- **LD2 (Load Data 2):** Saída que retorna o valor (32 bits) armazenado no registrador indicado em LR2 (Figura 13).

Figura 13 - Pino LD2



Fonte: Autoria própria, 2024.

- **RegWrite (Register Write):** Sinal para controlar a leitura ($\text{RegWrite} = 0$) ou escrita ($\text{RegWrite} = 1$) no registrador (Figura 14).

Figura 14 - Pino RegWrite



Fonte: Autoria própria, 2024.

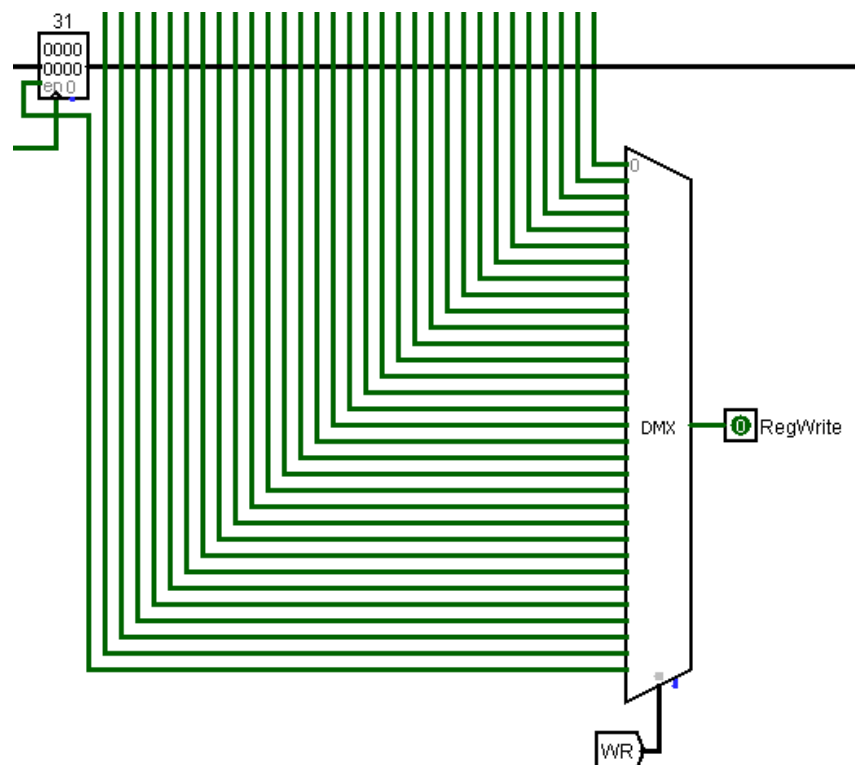
Ademais, adicionamos um clock para atualizar ou ler o valor ao gatilho e 32 registradores ordenados na vertical e numerados de 0 a 31 (Figura 17).

O banco de registradores recebe um valor em WR que indica o número de um registrador a receber um dado (WD). Portanto, para que fosse possível encontrar o registrador indicado e para direcionar o RegWrite para o mesmo registrador, foram adicionados dois demultiplexadores, cuja função é a de distribuir a entrada para a saída indicada. Além disso, para leitura dos dados contidos nos registradores indicados em LR1 e LR2 foram utilizados dois multiplexadores, cuja função é a de selecionar o registrador indicado e enviar para as saídas LD1 e LD2.

2.2.1. RegWrite

O pino de RegWrite tem a função de habilitar a escrita do registrador indicado em WR, por isso foi utilizado um demultiplexador que tem seus fios ligados aos botões de Enable dos registradores. Dessa forma, a saída selecionada em WR é direcionada ao registrador com o mesmo número, ou seja, o valor de RegWrite é vinculado à entrada do registrador que tem a função de tornar os gatilhos do clock efetivos ou não (Figura 15).

Figura 15 - Demultiplexador que direciona o RegWrite



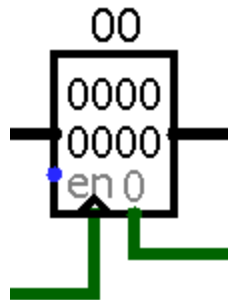
Fonte: Autoria própria, 2024.

2.2.2. Registrador 00

De acordo com o MIPS *reference card*, o registrador indicado pelo número “00” representa uma constante 0 que não pode permitir escrita. Portanto, para que isso fosse verificado

no banco de registradores, foi escolhida a opção de ligar o fio que se direciona ao registrador zero e sai do demultiplexador vinculado ao RegWrite com a região denominada Clear (“0”), a qual zera o registrador quando a entrada for diferente de 0 (Figura 16).

Figura 16 - Registrador 00

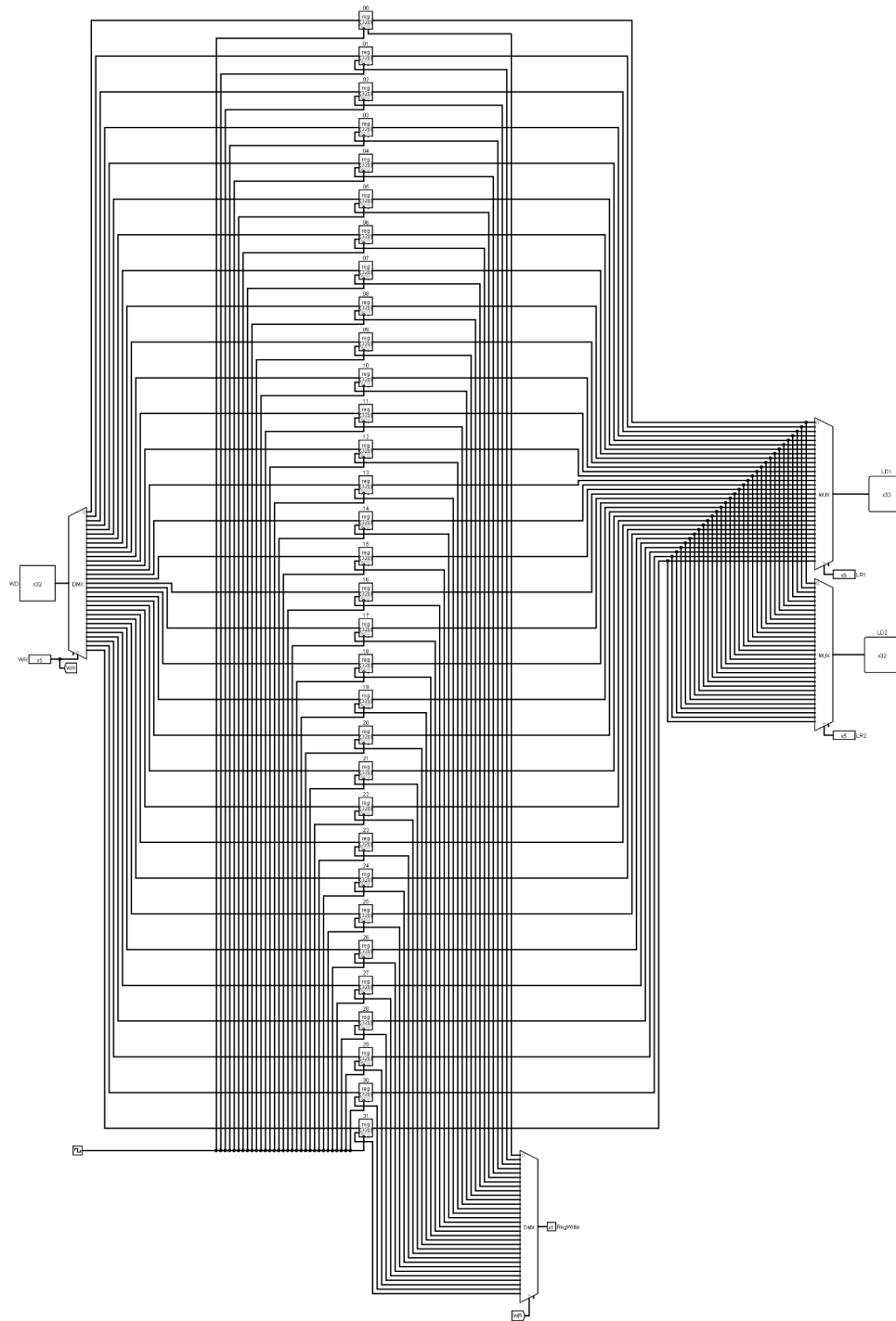


Fonte: Autoria própria, 2024.

2.2.3. Clock

O clock inserido no banco de registradores futuramente será vinculado a outras partes do *datapath*, mas sua função dentro desse componente é de sincronizar as ações de acesso aos registradores ao pulso do clock. Na figura 16, é possível verificar que existe um clock ligado a cada registrador na entrada marcada por um triângulo.

Figura 17 - Visão geral do banco de registradores

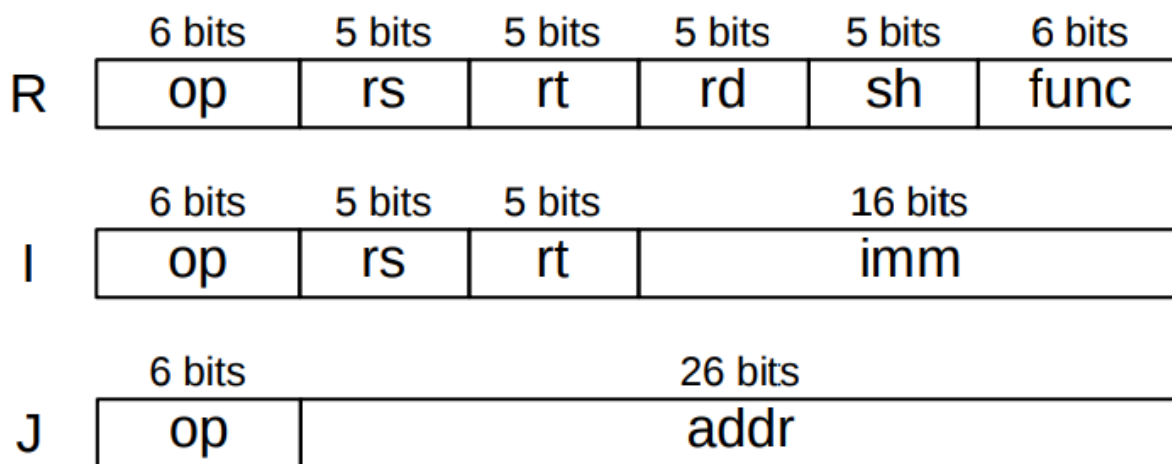


Fonte: Autoria própria, 2024.

2.3. Comportamento do decodificador

O decodificador tem a função de separar os bits de uma instrução nos formatos de cada um dos modos de endereçamento (R, I e J) em código de máquina em uma arquitetura MIPS (Figura 18).

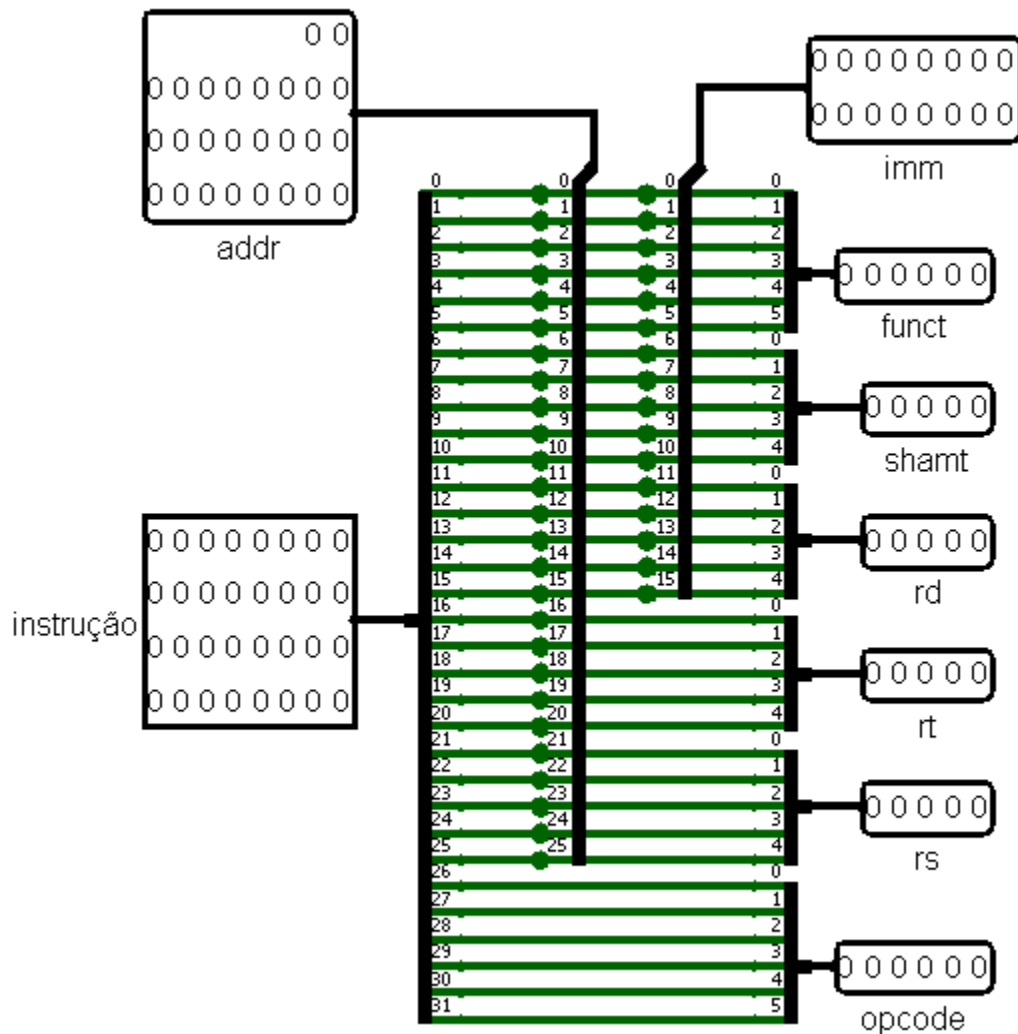
Figura 18 - Modos de endereçamento no MIPS



Fonte: MIPS Reference Card, 2024.

Esse componente não possui o papel de definir qual o tipo de endereçamento da instrução de entrada, por isso foram utilizados diversos distribuidores (Figura 19) que separam os bits da instrução de entrada (pino de instrução) para cada uma das partes dos formatos das instruções presentes na figura 18.

Figura 19 - Decodificador



Fonte: Autoria própria, 2024.

2.4. Comportamento da unidade de extensão de sinal

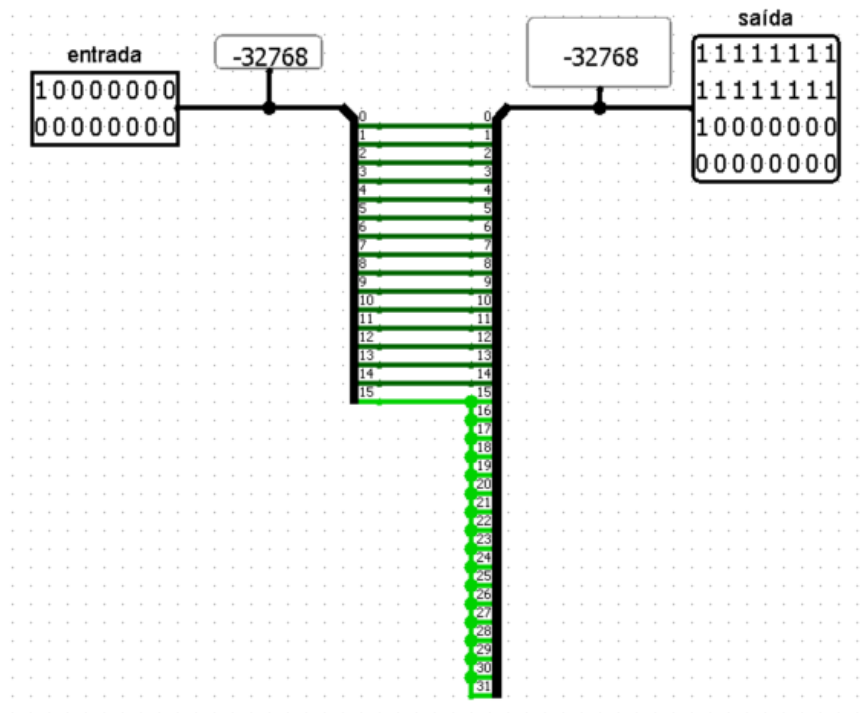
A unidade de extensão de sinal possui como finalidade transformar o valor de uma entrada de 16 bits em uma saída de mesmo valor, mas com formato de 32 bits. Esta atua de maneira similar ao extensor de bits do Logisim, colocando todos os n-1 bits do valor em suas

respectivas posições na saída, e replicando o bit mais significativo - e último - para todas as posições restantes.

Para garantir que o valor de um número binário seja preservado ao estender seu tamanho, as posições adicionais devem ser preenchidas de acordo com o bit mais significativo do número original. Se o último bit for 1, o número é negativo, e todas as posições adicionais devem ser preenchidas com 1. Isso mantém a representação em complemento de dois, que consiste em inverter os bits e adicionar 1 para obter o valor negativo correspondente.

Por outro lado, se o último bit for 0, o número é positivo, e as posições restantes devem ser preenchidas com 0, mantendo o valor original. Este método pode ser observado na Figura 20, que mostra tal implementação através da utilização de dois distribuidores no simulador Logisim.

Figura 20 - Unidade de extensão de sinal



Fonte: Autoria própria, 2024.

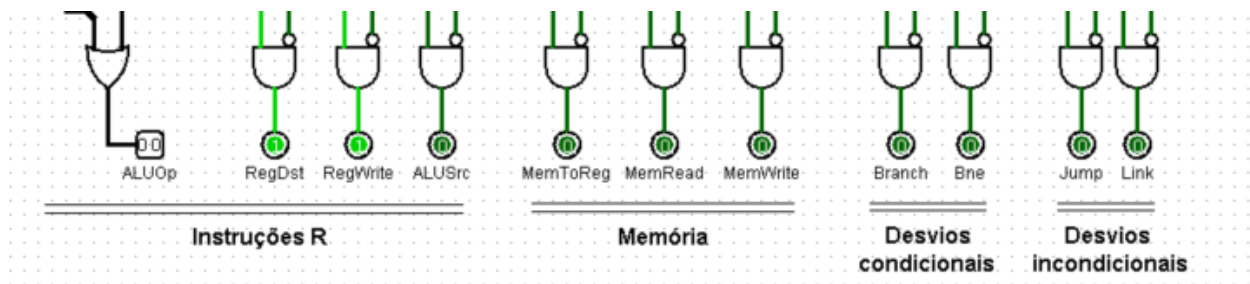
Ademais, a unidade de extensão de sinal é responsável, dentro do contexto do *datapath*, por transformar o valor da saída do decodificador imediato (*imm*), em 32 bits, visto que possui 16

inicialmente. Isto ocorre para que o dado possa ser processado na unidade lógica e aritmética, que segue o padrão do MIPS de entradas de 32 bits, possibilitando assim o processamento de instruções do tipo I como sw, lw e addi.

2.5. Comportamento da unidade de controle principal (UC)

A unidade de controle principal (UC) é responsável pela definição dos sinais de controle do processador, que determinam ações ao longo de todo o caminho de dados. Essa, através das entradas Opcode (6 bits) e JumpRegister - ou JR - (1 bit) indica quais são os valores dos seguintes sinais, suas saídas, dispostas na Figura 21.

Figura 21 - Sinais de controle da UC



Fonte: Autoria própria, 2025.

2.5.1. Sinais de controle

As saídas da UC, definidas como os sinais de controle do caminho de dados, determinam todas as operações de processamento de dados. Elas, também apresentadas na Figura 21, têm suas funções descritas abaixo:

- **ALUOP (2 bits):** Define a operação dentro da unidade de controle da ULA;
- **RegDst (1 bit):** Seleciona o registrador de destino (rt ou rd) para escrita;
- **RegWrite (1 bit):** Habilita a escrita no banco de registradores;

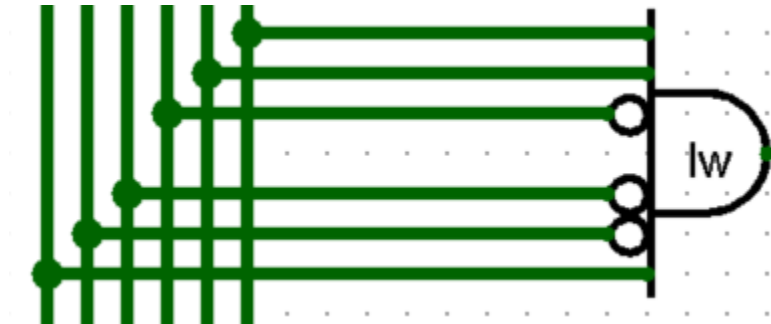
- **ALUSrc (1 bit):** Escolhe entre dado de registrador ou valor imediato (*imm*) como entrada da ULA;
- **MemToReg (1 bit):** Determina se o dado escrito no registrador vem da memória ou da ULA;
- **MemRead (1 bit):** Habilita leitura da memória de dados;
- **MemWrite (1 bit):** Habilita escrita na memória;
- **Branch (1 bit):** Controla o desvio condicional se os valores comparados forem iguais;
- **Bne (1 bit):** Controla o desvio condicional se os valores comparados forem diferentes;
- **Jump (1 bit):** Controla desvio incondicional;
- **Link (1 bit):** Indica se o endereço de retorno (PC+4) deve ser salvo.

2.5.2. Opcode

A entrada nomeada de Opcode é parte de cada uma das instruções da arquitetura MIPS, como pode ser observado na Figura 18 (representado como “op”). Essa é definida como os primeiros 6 bits de uma instrução, e através de seu valor, é possível discernir quando é necessário fazer desvios condicionais, incondicionais, escrita nas memórias e operações na ULA. As operações apresentadas (instruções tipo R, *addi*, *lw*, *sw*, *bew*, *bne*, *j* e *jal*) possuem valores de Opcode diferentes, tornando possível identificar sua ocorrência. Foram escolhidas para realizar tal separação uma sequência de portas AND, que possuem saída de valor 1 quando o valor do Opcode é o correspondente à instrução.

Por exemplo, para a instrução *LW*, que é selecionada apenas quando o Opcode é igual a 100011, foram utilizadas negações nas entradas dos bits da porta AND designada para tal, onde os bits são iguais a 0 - bits 2, 3 e 4 (Figura 22).

Figura 22 - Porta AND da instrução lw

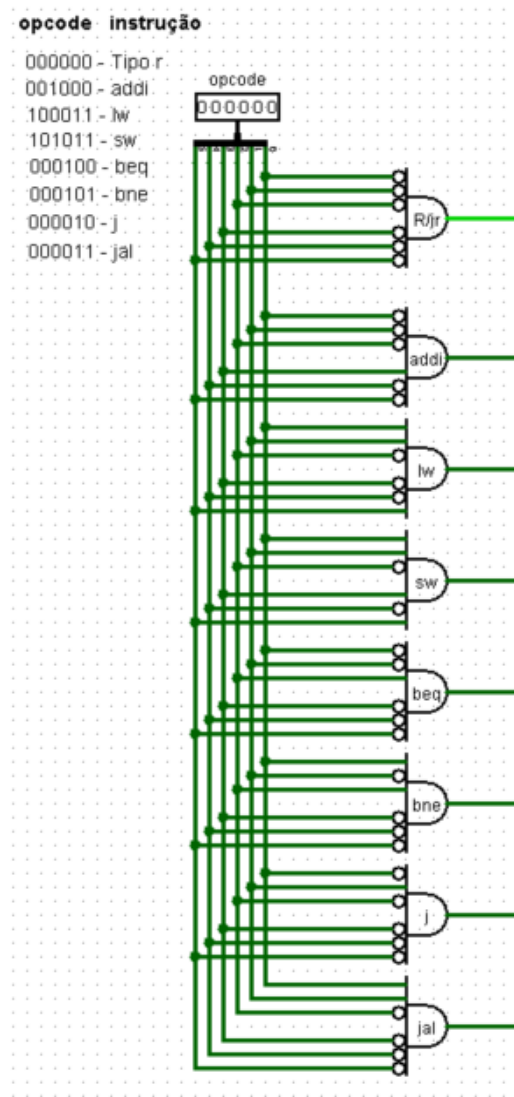


Fonte: Autoria própria, 2025.

Na Figura abaixo, é possível visualizar os códigos de cada uma das instruções, assim como as portas AND de cada uma das operações mencionadas anteriormente com as negações necessárias de modo a identificar qual instrução é a selecionada a partir da entrada Opcode.

Vale ressaltar que as instruções do tipo R possuem todas o mesmo Opcode (000000), diferenciando-se apenas pelo campo FUNCT. Isso significa que, embora a UC inicialmente identifique o tipo de instrução pelo Opcode para operações do tipo R, somente dentro da unidade de controle da ULA que torna-se possível determinar qual operação específica a ULA deve executar.

Figura 23 - Separação das instruções por Opcode

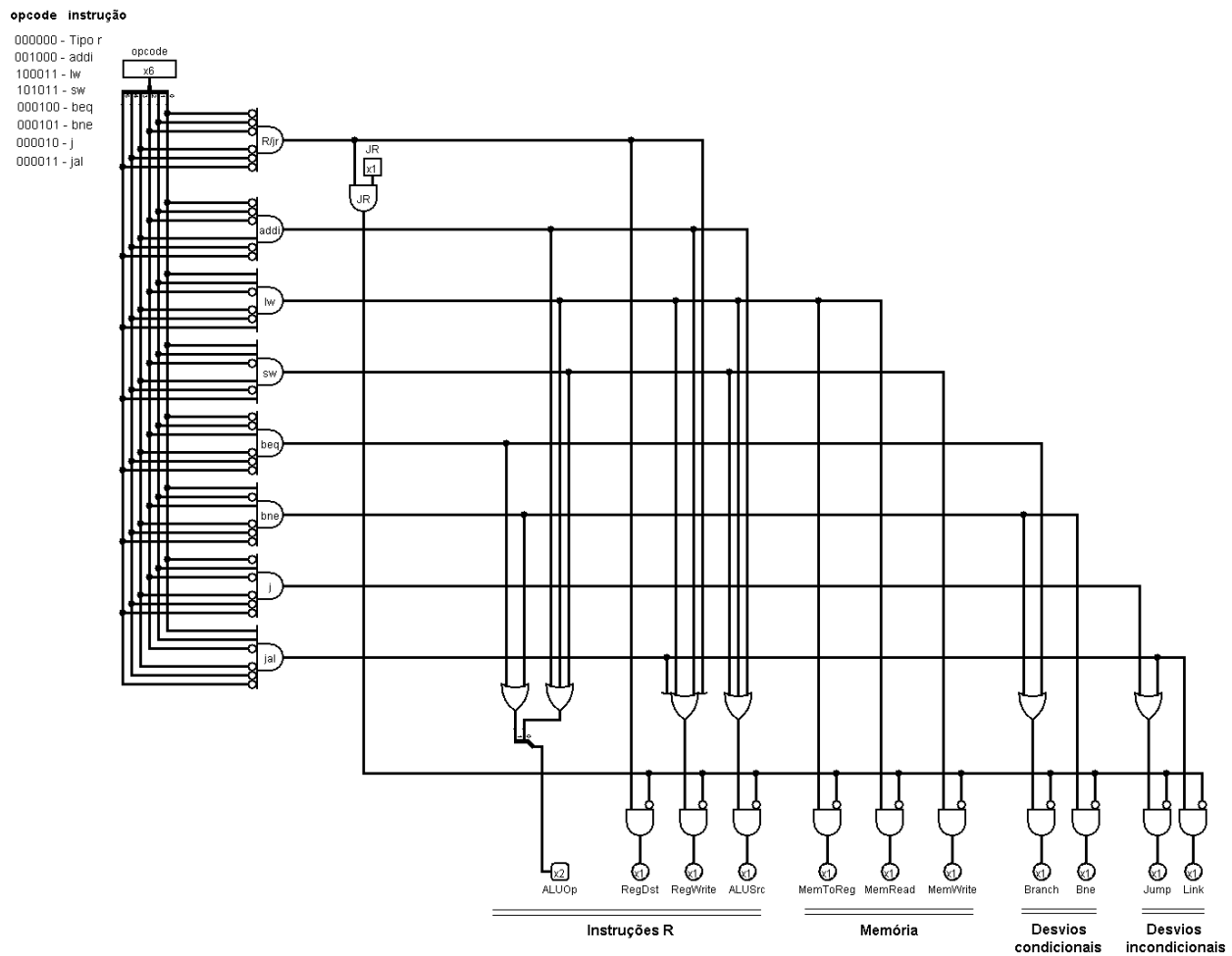


Fonte: Autoria própria, 2025.

Dessa forma, ao identificar a instrução utilizada, é possível determinar quais sinais devem ser ativados. Isso é feito por meio da conexão com portas lógicas OR, que agrupam todas as operações responsáveis por ativar cada saída. No caso da definição da saída ALUOp, a única de 2 bits, foram utilizados distribuidores do Logisim de modo a separar os valores que são utilizados na unidade de controle da ULA, e determiná-los separadamente de acordo com a seleção.

Adicionalmente, a entrada JR também possui sua importância por anular todos os sinais caso esteja ativada, indicando a ocorrência de uma instrução de salto incondicional JR junto ao Opcode 000000. De modo a desenvolver tal função, está ligada logo antes de todas as saídas (exceto ALUOp, que terá valor 00, indicando ser uma operação de Tipo R), desativando todas no caso de sua ativação.

Figura 24 - Circuito da unidade de controle principal (UC)



Fonte: Autoria própria, 2025.

Portanto, de modo simplificado, o Quadro abaixo mostra quais são as instruções diferenciadas dentro da unidade de controle de acordo com Opcode e JR, e quais os valores dos sinais de controle de acordo com a implementação para cada uma.

Quadro 1 - Instruções do MIPS e valores dos sinais de controle

Instrução	Op code	ALU Op	Reg Dst	Reg Write	ALU Src	Mem To Reg	Mem Read	Mem Write	Branch	Bne	Jump	Link
Tipo R	000 000	00	1	1	0	0	0	0	0	0	0	0
addi	001 000	01	0	1	1	0	0	0	0	0	0	0
lw	100 011	01	0	1	1	1	1	0	0	0	0	0
sw	101 011	01	0	0	1	0	0	1	0	0	0	0
beq	000 100	10	0	0	0	0	0	0	1	0	0	0
bne	000 101	10	0	0	0	0	0	0	1	1	0	0
j	000 010	00	0	0	0	0	0	0	0	0	1	0
jal	000 011	00	0	1	0	0	0	0	0	0	1	1
jr	000 000	11	0	0	0	0	0	0	0	0	0	0

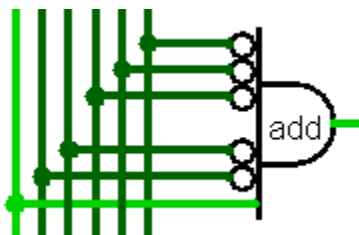
Fonte: Autoria própria, 2025.

2.6. Comportamento da unidade de controle da ULA

O elemento denominado por unidade de controle da ULA desempenha o papel de definir a operação da ULA para cada uma das instruções. Para cumprir essa função, é necessário que ela possua dois pinos de entrada: ALUOp (2 bits) e funct (6 bits).

De acordo com os modos de endereçamento da arquitetura MIPS (Figura 18), apenas as instruções do tipo R possuem o campo funct, o qual é utilizado para selecionar as operações da ULA realizadas por cada instrução desse tipo. Por isso, para cada uma das instruções aritméticas e lógicas que coincidem com as operações da ULA (and, or, nor, xor, slt, add, sub, sll e srl) foram criadas portas AND com o objetivo de fazer com que a saída seja 1 apenas para a instrução selecionada no pino funct. Por exemplo, para a AND que deve ter a saída 1 apenas quando a instrução do tipo add for selecionada, foram utilizadas negações nas entradas dos bits da função de and (100000) que fossem iguais a 0, ou seja, todos menos o bit mais significativo (Figura 25).

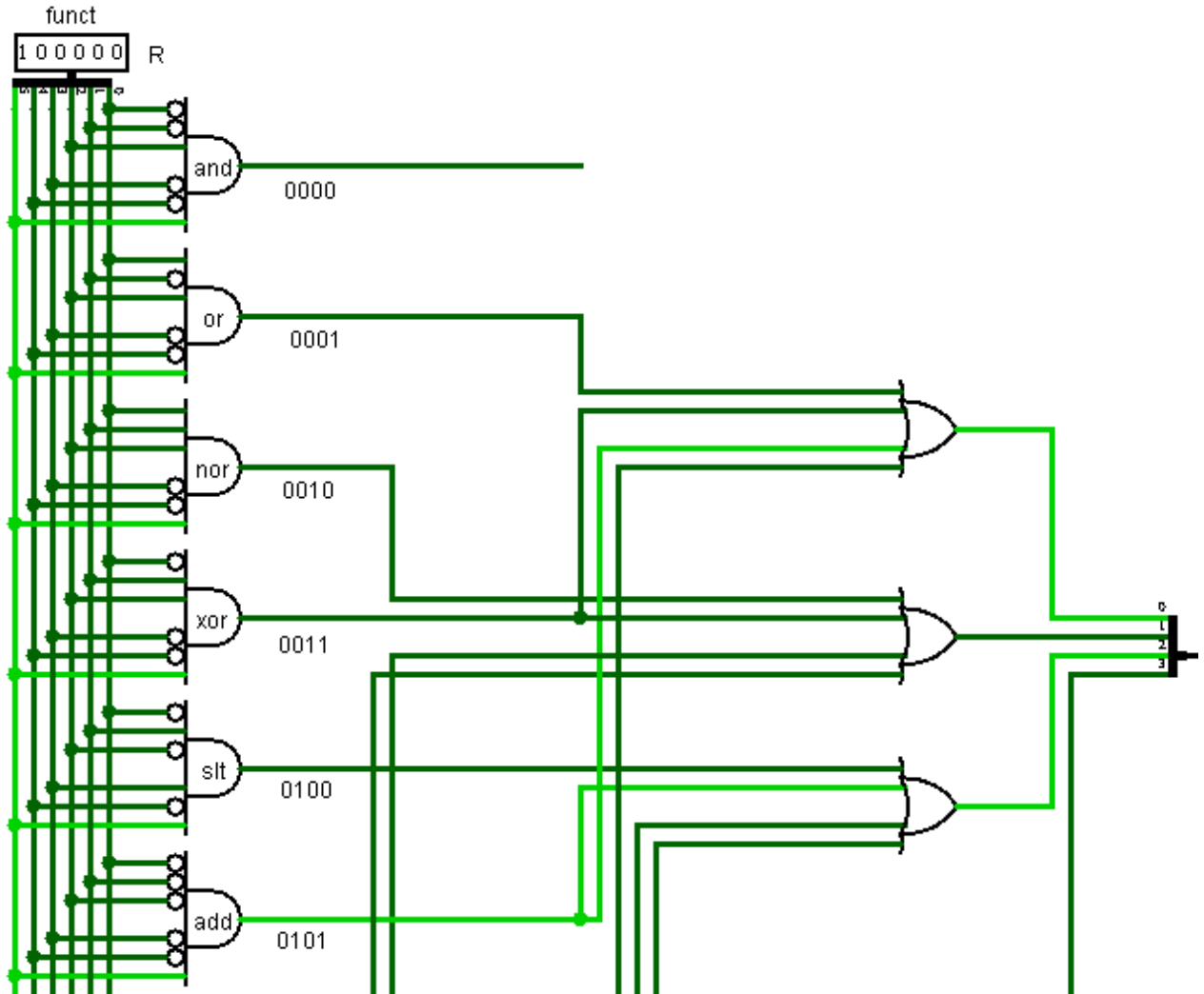
Figura 25 - Porta and da instrução add



Fonte: Autoria própria, 2024.

Após a indicação da instrução lógica e aritmética selecionada, foi necessário converter a saída em um pino que coincidissem com uma das operações da ULA mencionadas na seção 2.1. Para tal, foram criadas portas OR que, para cada um dos 4 bits do distribuidor, tivessem como entrada apenas as instruções que alterassem o bit de destino da OR. Exemplificando, a instrução add deve selecionar a operação 0101 (soma) da ULA, ou seja, ela deve tornar o bit 0 e o bit 2 sinais ativos (1). Com esse objetivo a saída da porta AND que só produz um resultado verdadeiro para a instrução add foi ligada apenas às portas OR com destino aos bits 0 e 2.

Figura 26 - Seleção da instrução add gerando uma saída 0101



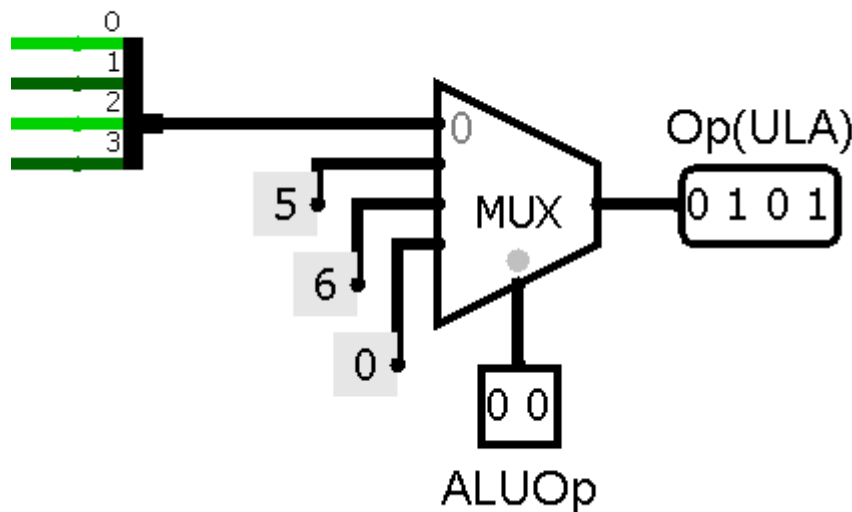
Fonte: Autoria própria, 2024.

Entretanto, como apenas as instruções do tipo R possuem o campo funct, foi fundamental utilizar o pino ALUOp que provem da unidade de controle principal a fim de selecionar as operações da ALU para instruções com outros modos de endereçamento. Como ALUOp possui dois bits, utilizou-se um multiplexador com dois bits para seleção (Figura 27).

- **Entrada 00:** Seleciona a saída do distribuidor com a operação para instruções do tipo R.

- **Entrada 01:** Visto que o valor 01 para ALUOp deve selecionar a operação de soma (0101), essa entrada foi ligada a uma constante de quatro bits com o valor 0x5 em hexadecimal.
- **Entrada 10:** Análoga à entrada anterior, essa deve selecionar a operação de subtração (0110), por isso foi ligada a uma constante de quatro bits com o valor 0x6 em hexadecimal.
- **Entrada 11:** Considerando que as instruções contempladas no *datapath* implementado não utilizam outro tipo de operação, essa entrada foi ligada a uma constante de quatro bits com o valor 0x0 em hexadecimal.

Figura 27 - Multiplexador que seleciona a operação a ser introduzida na ULA

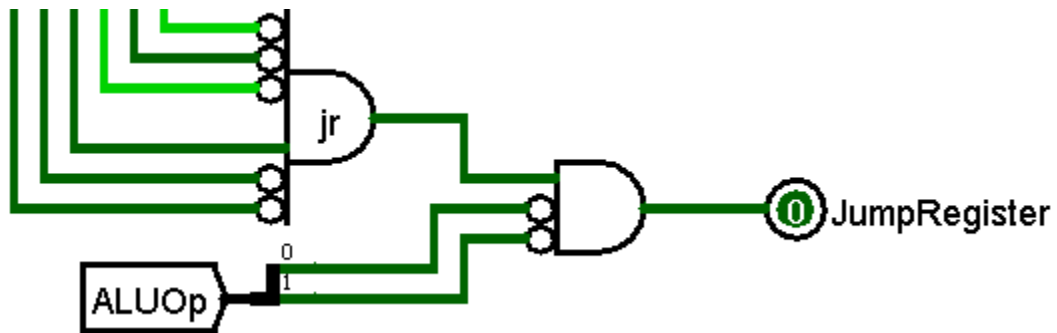


Fonte: Autoria própria, 2024.

2.6.1. JumpRegister

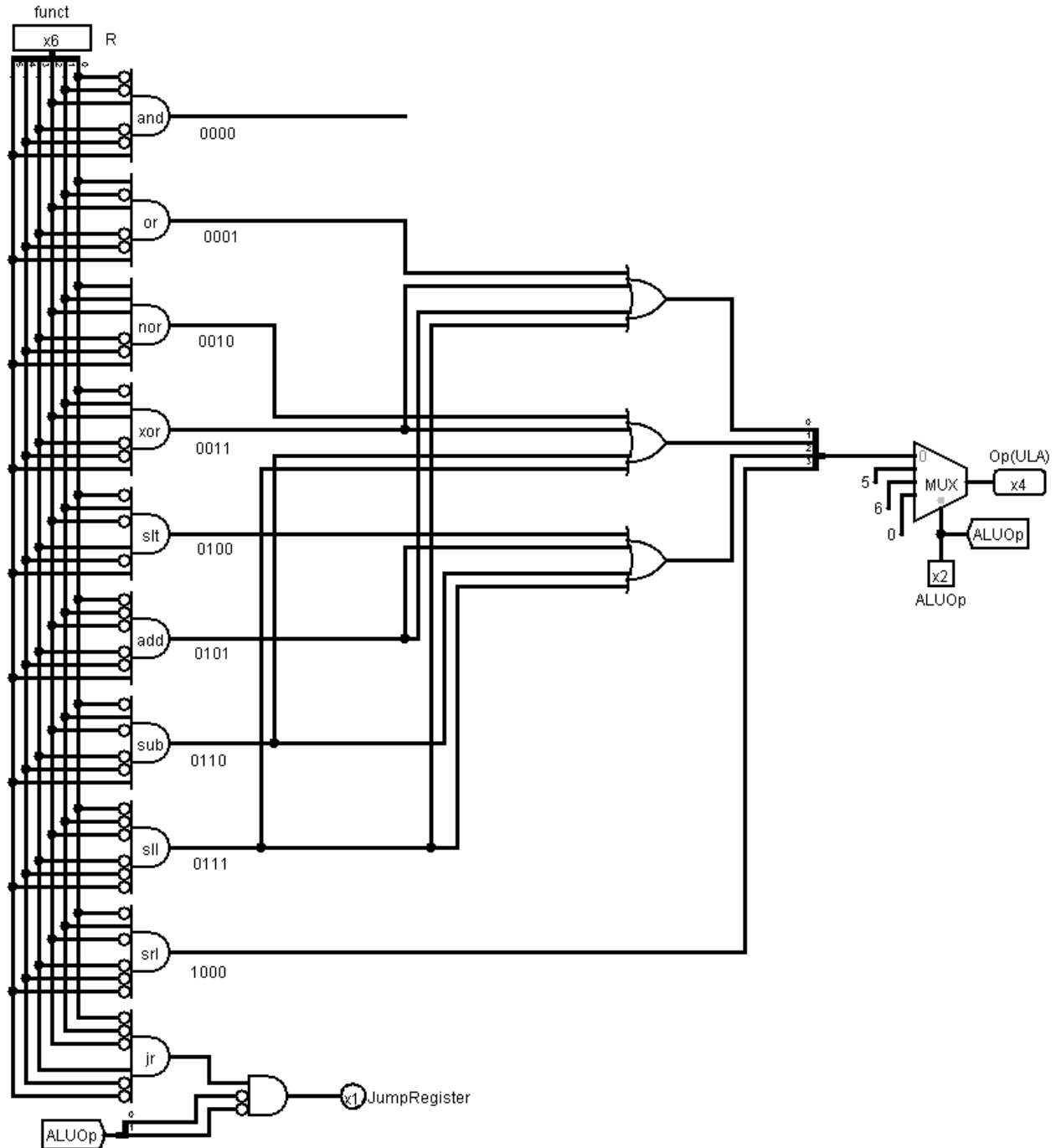
Ademais, tendo em vista que a instrução jr possui o mesmo Opcode das instruções do tipo R, para diferenciá-la utilizou-se uma porta and que produzisse saída ativa apenas para a função de jr (001000), essa saída foi ligada a uma porta and que também possui como entrada os bits negados de ALUOp (que é 00 para esse tipo de instrução). Dessa forma, nenhuma outra instrução que termine em 1000 e não seja do tipo R ativa o pino do JumpRegister, que terá o valor 1 para a instrução jr.

Figura 28 - JumpRegister



Fonte: Autoria própria, 2024.

Figura 29 - Visão geral da unidade de controle da ULA



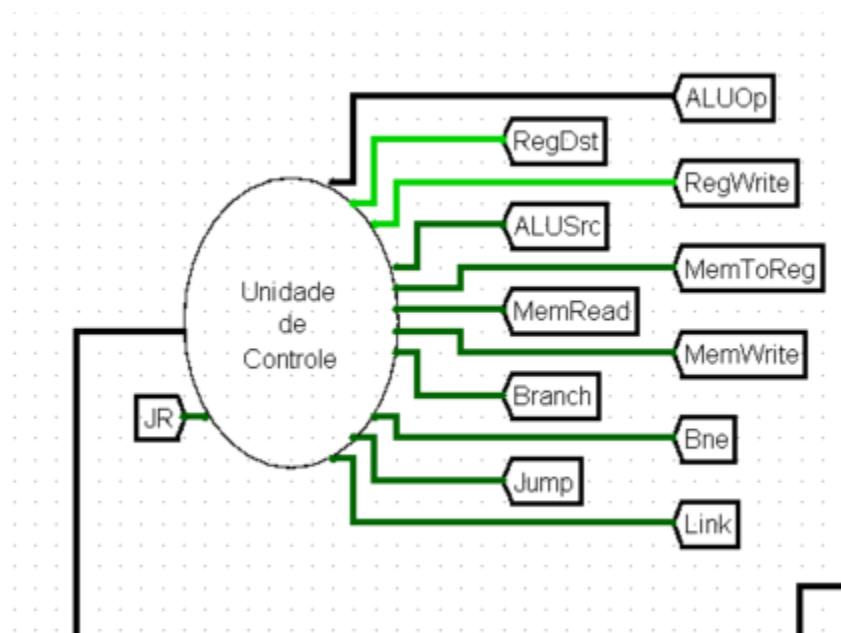
Fonte: Autoria própria, 2024.

2.7. Multiplexadores e sinais de controle

Ao se tratar de um *datapath* monociclo, multiplexadores servem como um ponto essencial para direcionar os dados entre diferentes componentes, como registradores e memória, enquanto os sinais de controle são fundamentais para gerenciar o fluxo de dados e as operações neste, garantindo a execução correta de instruções.

Os sinais de controle, como mencionado anteriormente, são as saídas da unidade de controle principal, determinadas a partir do Opcode de cada instrução. Na figura abaixo estão dispostos os sinais, que tiveram túneis do Logisim interligados a cada conexão de modo a simplificar as ligações ao longo do circuito de forma mais organizada.

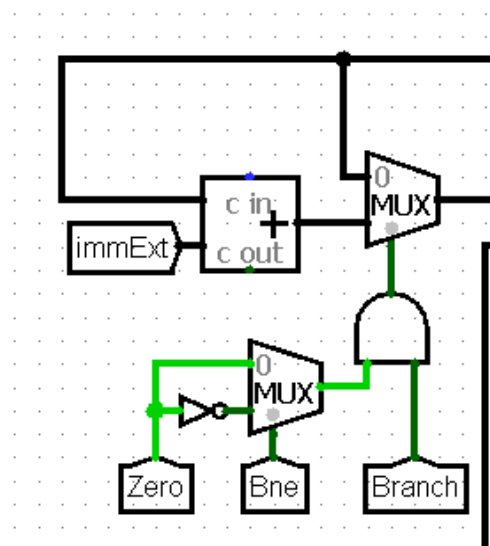
Figura 30 - Túneis dos sinais de controle



Fonte: Autoria própria, 2025.

Estes, por sua vez, são conectados a outros componentes ao longo do circuito, como os multiplexadores, com os quais é possível determinar entre duas entradas aquela que será a saída.

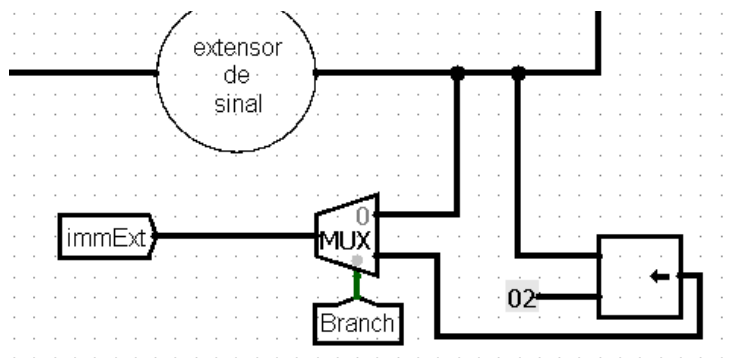
Figura 31 - Multiplexadores de desvio condicional



Por sua vez, o túnel immExt também presente na Figura 31, representa o valor de endereço relativo a ser utilizado na próxima instrução do registrador PC quando a atual é uma de desvio condicional (branch). Isso porque nesse tipo de instrução o endereço de destino é calculado a partir do endereço da instrução atual mais o valor do imediato da instrução que indica quantas palavras (4 bytes) a partir de PC+4 o contador de programa deve pular. Desta forma, o

valor de imediato deve ser deslocado 2 bits à esquerda, multiplicando-o por 4, conforme ilustrado na Figura 32 abaixo.

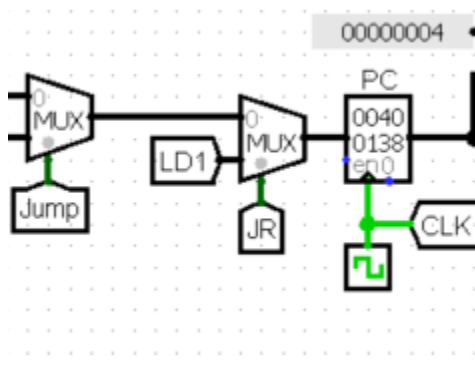
Figura 32 - Deslocador de imediato



Fonte: Autoria própria, 2025.

Já na Figura 33 estão representados os multiplexadores responsáveis por determinar se o caminho de execução irá seguir o fluxo usual (estando o primeiro interligado na saída do último multiplexador da Figura 31) ou se um “Jump” ou “JR” serão realizados. Desta forma, se uma instrução de salto for executada, o processador atualizará o PC para o endereço especificado na instrução de desvio - que está contido em \$ra para as instruções do tipo JR (LD1) e na saída do concatenador para instruções do tipo J -, ignorando os valores anteriores.

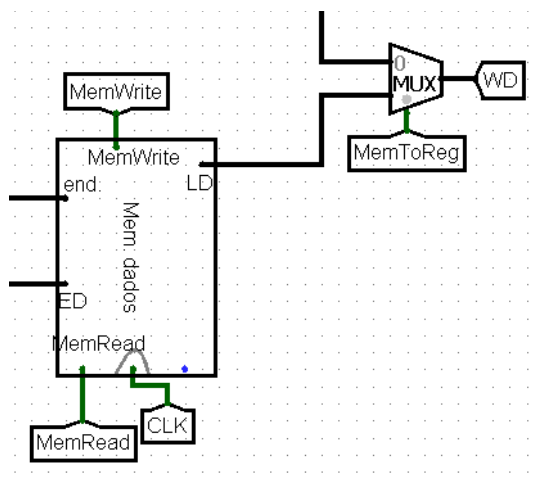
Figura 33 - Multiplexadores de desvio incondicional



Fonte: Autoria própria, 2025.

A Figura 34 ilustra o uso de um multiplexador com duas entradas, sendo uma o resultado da ULA e a outra o registrador lido na memória de dados. A saída deste é determinada pelo sinal de controle MemToReg, que ao ser levantado indica a gravação de um valor da memória em um registrador do banco de registradores (WD). Desta maneira, seu uso auxilia na definição da instrução, uma vez que caso o dado a ser salvo venha da memória, o sinal é 1 e o valor de saída do multiplexador vem de LD na memória de dados. Caso contrário, o valor do sinal MemToReg é 0 e o dado registrado no banco de registradores é o resultado das operações da ULA.

Figura 34 - Multiplexador de escrita



Fonte: Autoria própria, 2025.

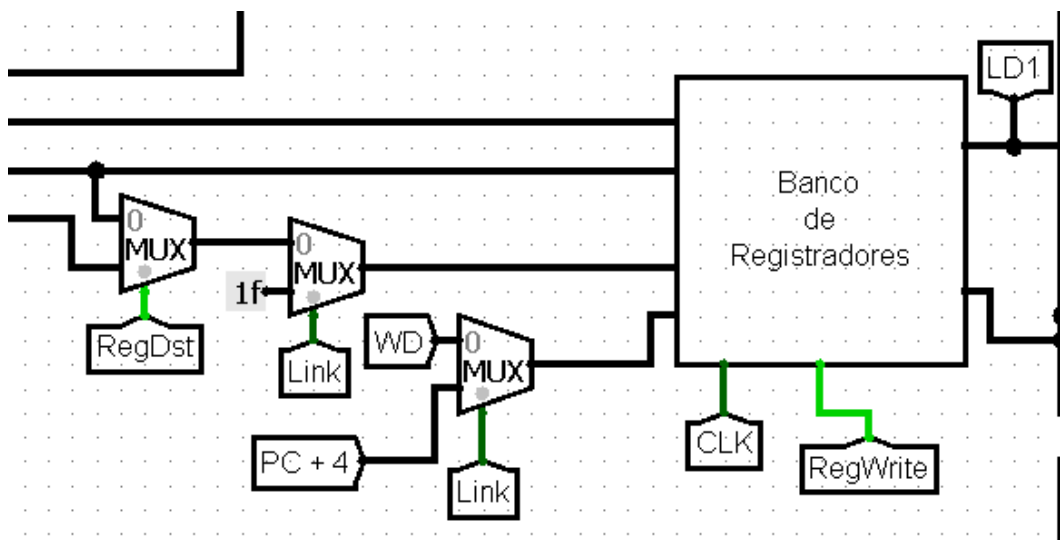
Ainda, na Figura 35 estão dispostos os multiplexadores utilizados para determinar quais serão as entradas WD (dado a ser escrito) e WR (registrador destino) no banco de registradores. O primeiro (da esquerda para a direita), que possui como operador o sinal de controle RegDst determina se o registrador a ser escrito será o determinado pelo campo rt (para instruções do Tipo I) ou pelo rd (para instruções do Tipo R), provenientes do decodificador de instruções.

O segundo define se o registrador a ser escrito será aquele selecionado no multiplexador anterior, ou o registrador de número 31 (0x1f em hexadecimal), o \$ra. O controle desse

multiplexador é feito pelo sinal Link, que ao ser ativado permite a escrita de PC+4 como endereço de retorno. Assim, torna-se possível o regresso após as chamadas de funções realizadas com a instrução jal, que também ocorre com o auxílio do último multiplexador.

Esse último, por sua vez, é o que decreta se o dado a ser escrito é oriundo da memória de dados, de onde vem o túnel WD como mostrado anteriormente, ou do endereço após o da instrução atual. Esse valor seria salvo no registrador \$ra como a linha de retorno e utilizado na saída de funções com a instrução jr \$ra.

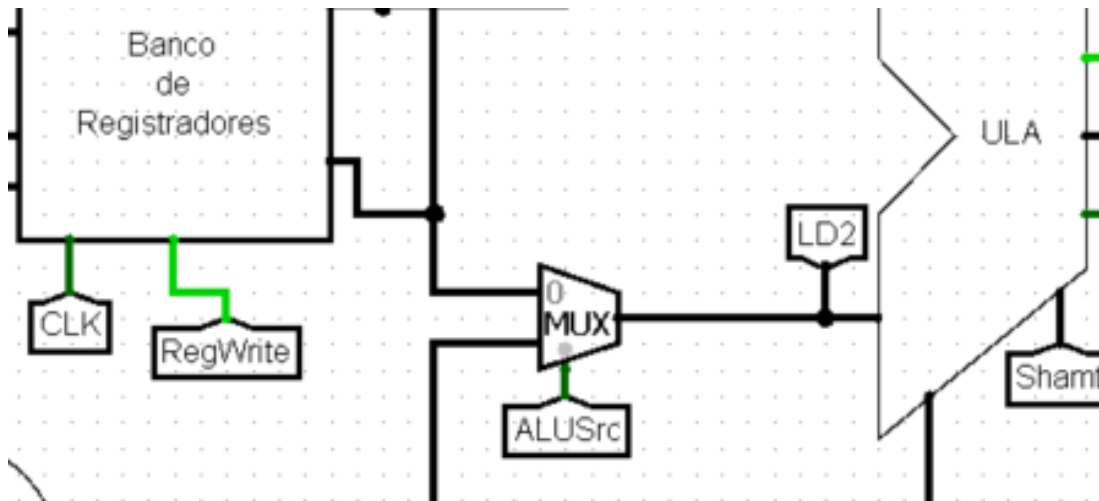
Figura 35 - Multiplexadores de WR e WD



Fonte: Autoria própria, 2025.

Por fim, o multiplexador representado na Figura abaixo é utilizado de modo a definir se a segunda entrada da ULA (B) recebe a saída LD2 do banco de registradores ou o valor imediato do campo imm passado pelo extensor de sinal, com 32 bits. De modo a escolher a operação, o sinal de controle ALUSrc é estabelecido como 0 para instruções que não usam imediato e 1 para as que usam.

Figura 36 - Multiplexador da entrada B da ULA

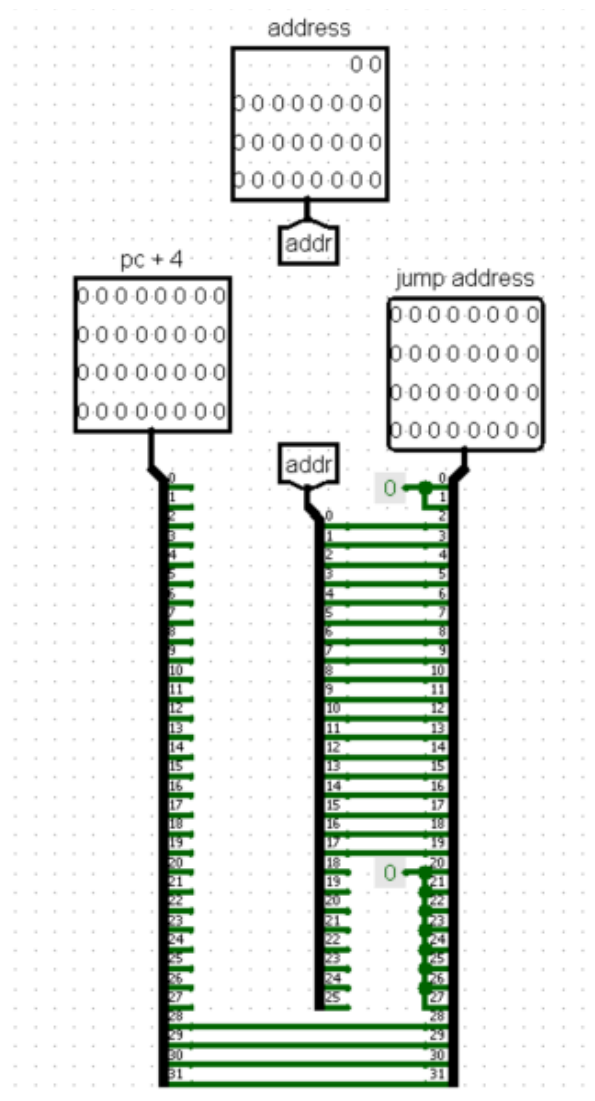


Fonte: Autoria própria, 2025.

2.8. Concatenador para instruções de desvio

O concatenador para instruções de desvio é mais um dos componentes que garante a execução de operações de desvio incondicional dentro do circuito. Sua construção envolve a integração de bits do endereço de desvio contido na instrução (addr, address), combinando-o com os 4 bits mais significativos de PC+4 - os quais indicam o segmento de memória do salto - de modo a redefinir a instrução seguinte. O pino addr também precisa ser multiplicado por quatro (deslocado duas vezes à esquerda) para convertê-lo de palavras para bytes. Assim, é possível estabelecer o endereço final do desvio (jump address), como presente na Figura 35, e direcionar o circuito para a instrução correta.

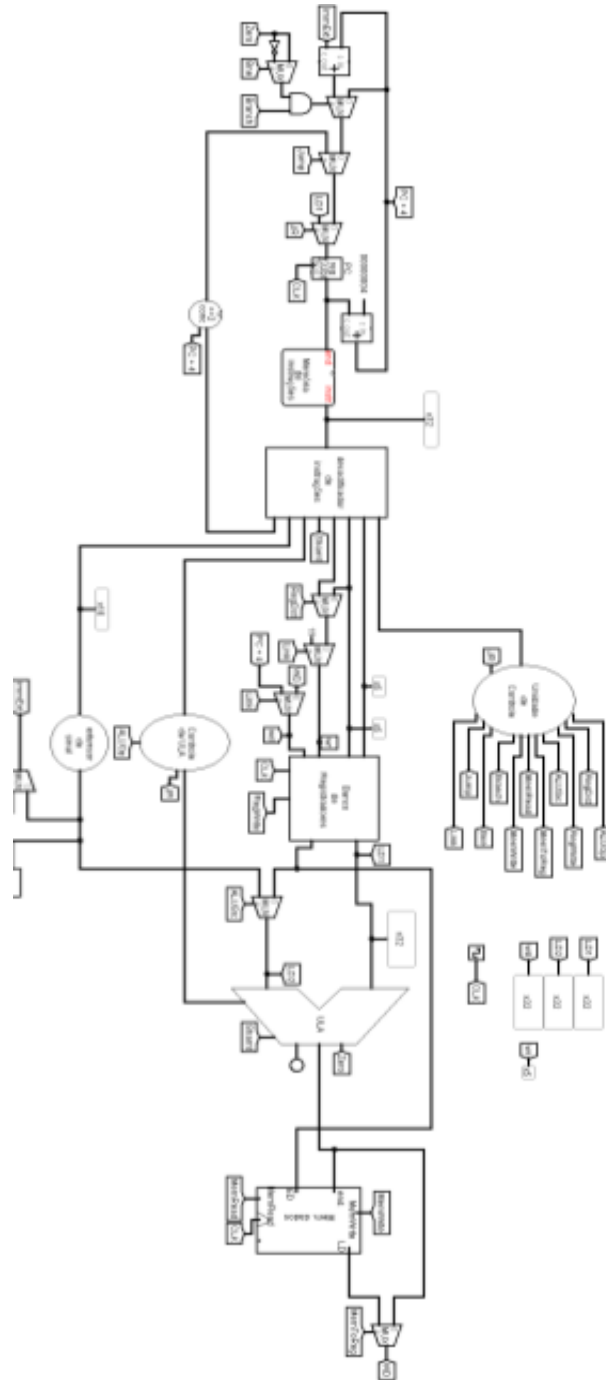
Figura 37 - Concatenador



Fonte: Autoria própria, 2025.

2.9. Datapath completo

Figura 38 - Simulador do caminho de dados completo, rotacionado 90° no sentido horário



Fonte: Autoria própria, 2025.

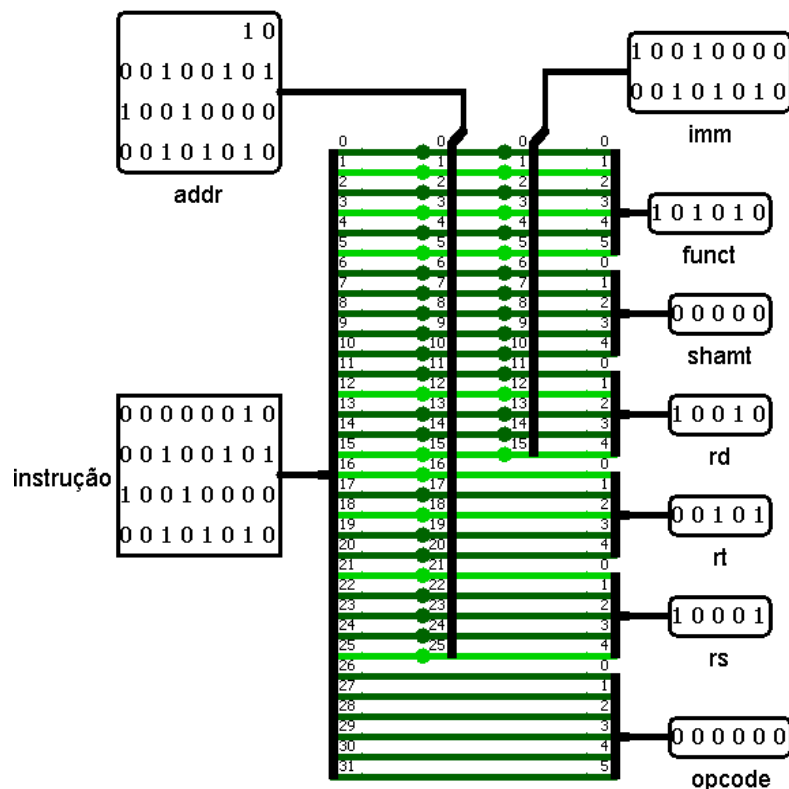
3. Manual de funcionamento

Posto que o *datapath* possui a função de interpretar as instruções e executar as operações especificadas, para facilitar a compreensão do seu funcionamento passo a passo será realizada a descrição da instrução `slt $s2, $s1, $a1` que possui os valores 0 em \$s1 e 6 em \$a1.

3.1. Instrução 1: `slt $s2, $s1, $a1`

1. O endereço contido em PC aponta para a região da memória de instruções no qual essa instrução pertence.
2. A memória de instruções retorna a instrução nesse endereço em hexadecimal, que é igual a 0225902a.
3. Essa instrução em hexadecimal é levada ao decodificador, o qual divide os bits para cada uma das divisões dos modos de endereçamento (Figura 38).

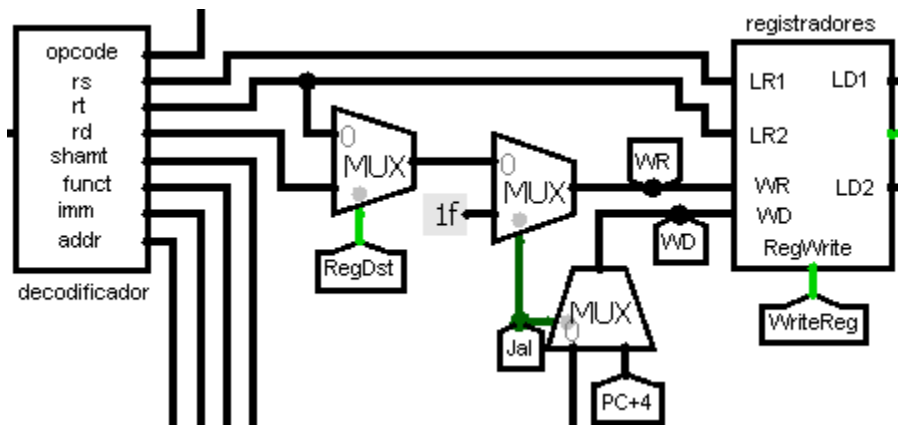
Figura 39 - Decodificador para a instrução `slt $s2, $s1, $a1`



Fonte: Autoria própria, 2025.

4. O campo opcode é direcionado para a unidade de controle principal onde são definidos os sinais de controle, uma vez que a instrução é do tipo R ela ativa os sinais RegDst e RegWrite e define o valor 00 para ALUOp.
5. O registrador indicado em rs (\$s1 = 10001) vai para a entrada LR1 do banco de registradores, visando ler o dado de registrador e retorná-lo em LD1.
6. A saída rt (\$a1 = 00101) é direcionada para a entrada LR2 do banco de registradores, visando ler o dado de registrador e retorná-lo em LD2.
7. Para definir a entrada de WR do banco, ou seja, o registrador no qual um dado vai ser escrito, o RegDst define a entrada 1 que é o registrador rd (\$s2 = 10010) e, como o sinal de controle Jal não foi ativado, essa é a entrada de WR (Figura 38).

Figura 40 - Definição das entradas do banco de registradores para a instrução slt \$s2, \$s1, \$a1



Fonte: Autoria própria, 2025.

8. Os campos imm, addr e shamt não são utilizados nessa instrução.
9. O campo funct é direcionado para a unidade de controle da ULA a fim de selecionar a operação a ser executada na ULA. O campo funct dessa instrução junto com seu ALUOp definido anteriormente resulta na operação 0100 (slt) como saída desse componente.
10. Os dados lidos do banco de registradores são enviados como operadores da ULA, uma vez que ALUSrc não está ativado para selecionar o valor do imediato (imm) como

segundo operando. A operação `slt` da unidade lógica e aritmética resulta na saída -1, dado que o valor contido em `LD1` é menor que o valor contido em `LD2`.

11. O resultado da ULA é direcionado para a memória de dados - que não utiliza os valores de entrada por não possuir seus sinais de controle ativos (`MemWrite` e `MemRead`) - e para a entrada `WD` do banco de registradores, visto que os sinais `MemToReg` e `Jal` selecionam as entradas 0 dos multiplexadores.
12. Como nenhum dos sinais que sinalizam instruções de desvio estão ativos, no próximo clock a instrução a ser executada será `PC+4` e o registrador `$s2` terá o valor -1 armazenado.

4. Instruções do *datapath*

4.1. Linguagem de alto nível (C)

```
int produto(int a, int b) {
    int produto = 0;
    for (int i = 0; i < b; i++) {
        produto += a;
    }
    return produto;
}

int calcular_soma_dos_produtos_pares(int inicio, int fim) {
    int soma = 0;
    for (int i = inicio; i <= fim; i++) {
        for (int j = inicio; j <= fim; j++) {
            int prod = produto(i, j);
            int modulo = prod;
            while (modulo >= 2) {
                modulo -= 2;
            }
            if (modulo == 0) {
                soma += prod;
            }
        }
    }
}
```

```
    return soma;
}
int main() {
    int resultado = calcular_soma_dos_produtos_pares(6, 7);
}
```

4.2. Assembly (MIPS)

```
j main
produto:
    addi $sp, $sp, -12
    sw $s0, 0($sp) #produto
    sw $s1, 4($sp) #i
    sw $s2, 8($sp) #bool

    and $s0, $zero, $zero # int produto = 0
    and $s1, $zero, $zero # int i = 0
    for_produto:
        slt $s2, $s1, $a1 #i<b
        beq $s2, $zero, end_for_produto
        add $s0, $s0, $a0 #produto += a
        addi $s1, $s1, 1 #i++
        j for_produto

    end_for_produto:
        add $v0, $s0, $zero
        lw $s0, 0($sp)
        lw $s1, 4($sp)
        lw $s2, 8($sp)
        addi $sp, $sp, 12
        jr $ra #retorna para a função de soma

calcular_soma_dos_produtos_pares:
    addi $sp, $sp, -44
    sw $s0, 0($sp) #s0 = soma
    sw $s1, 4($sp) #s1 = i
    sw $s2, 8($sp) #s2 = j
    sw $s3, 12($sp) #s3 = prod
    sw $s4, 16($sp)
```

```
sw $s5, 20($sp)
sw $s6, 24($sp) #modulo
sw $s7, 28($sp)
sw $ra, 32($sp)
sw $a0, 36($sp) #inicio
sw $a1, 40($sp) #fim

add $s0, $zero, $zero # int soma = 0
add $s1, $zero, $a0 # i = inicio
addi $t2, $a1, 1 # fim+1
for_i:
slt $s4, $s1, $t2 # i<fim
beq $s4, $zero, end_for_i
add $s2, $zero, $a0 # j = inicio
    for_j:
        slt $s5, $s2, $t2 # j<fim
        beq $s5, $zero, end_for_j
        add $a0, $s1, $zero
        add $a1, $s2, $zero #produto(i, j)
        jal produto

        lw $a0, 36($sp)
        lw $a1, 40($sp)
        add $s3, $v0, $zero # prod = produto(i, j)

        add $s6, $zero, $s3 # modulo = prod
        while:
            addi $t0, $zero, 2
            addi $t1, $zero, 1
            slt $s7, $t1, $s6 # 1 < modulo
            beq $s7, $zero, end_while
            sub $s6, $s6, $t0 # modulo -= 2
            j while
        end_while:
            bne $s6, $zero, else # if modulo == 0
            add $s0, $s0, $s3 # soma += modulo
        else:
            addi $s2, $s2, 1 #j++
            j for_j
    end_for_j:
    addi $s1, $s1, 1 #i++
    j for_i
end_for_i:
```

```
    add $v0, $s0, $zero

    lw $s0, 0($sp)
    lw $s1, 4($sp)
    lw $s2, 8($sp)
    lw $s3, 12($sp)
    lw $s4, 16($sp)
    lw $s5, 20($sp)
    lw $s6, 24($sp)
    lw $s7, 28($sp)
    lw $ra, 32($sp)
    addi $sp, $sp, 44
    jr $ra # retorna main
main:
    addi $a0, $zero, 6
    addi $a1, $zero, 7
    jal calcular_soma_dos_produtos_pares
    add $s0, $v0, $zero
end:
j end
```

4.3. Linguagem de máquina (MIPS)

```
08100045
23bdfbf4
afb00000
afb10004
afb20008
00008024
00008824
0225902a
12400003
02048020
22310001
08100007
02001020
8fb00000
8fb10004
8fb20008
```

```
23bd000c
03e00008
23bdf fd4
afb00000
afb10004
afb20008
afb3000c
afb40010
afb50014
afb60018
afb7001c
afbf0020
afa40024
afa50028
00008020
00048820
20aa0001
022aa02a
12800016
00049020
024aa82a
12a00011
02202020
02402820
0c100001
8fa40024
8fa50028
00409820
0013b020
20080002
20090001
0136b82a
12e00002
02c8b022
0810002d
16c00001
02138020
22520001
08100024
22310001
08100021
02001020
8fb00000
```

8fb10004
8fb20008
8fb3000c
8fb40010
8fb50014
8fb60018
8fb7001c
8fbf0020
23bd002c
03e00008
20040006
20050007
0c100012
00408020
08100049

4.4. Tabela de pontuação

Tabela 1 - Pontuação das instruções escolhidas

Tipo		Pontos	Máximo	Quantidade	Valores	Total
Instruções	Instruções R (exceto JR)	4	20	4	16	75 (70)
	Instruções aritméticas com imediato	5	15	1	5	
	Desvios condicionais	7	14	2	14	
	Desvios incondicionais	8	24	3	24	
	Operações de memória	8	16	2	16	
Código	Estrutura condicional	2	4	1	2	38 (30)
	Laços de repetição não aninhados	4	8	2	8	
	Laços de repetição aninhados	8	8	1	8	
	Chamada de função folha	8	8	1	8	
	Chamada de função não-folha	12	12	1	12	

Fonte: Autoria própria, 2025.

5. Considerações finais

A reprodução de um *datapath* na prática facilita muito a compreensão do funcionamento do caminho de dados. Com a conclusão do projeto após sua divisão em três partes, tornou-se possível observar que a implementação dos circuitos de cada um dos componentes foi fundamental para fixar a ideia de suas atuações no *datapath*. A trajetória de construção de cada circuito evidenciou erros de interpretação da lógica por trás de cada saída que puderam ser solucionados com testes visando comprovar a eficácia dos circuitos. Finalmente, percebeu-se a complexidade deste fluxo de processamento ao unir todos os elementos constituídos anteriormente e fazê-los funcionar em uma singular e harmônica estrutura, possibilitando a execução de instruções dos tipos R, I e J.

6. Referências

- [1] PATTERSON, David A. HENNESSY, John L. **Organização e Projeto de Computadores**. Disponível em: Minha Biblioteca, (5a. edição). Grupo GEN, 2017. Acesso em: 7 dez. 2024.
- [2] **Mips reference card**. Disponível em: <https://www.cburch.com/cs/330/reading/mips-ref.pdf>. Acesso em: 7 dez. 2024.